

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf\_e5140909-890\agent-a2ceb083e05ae95f6.jsonl`
- SHA-256: `f774c10c9b4c0ac1e0301b175c594e6863b831b7c3f715a2766de9c654cf666b`
- Source modified: `2026-06-18T07:57:02+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_e5140909-890`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T07:54:35.119Z)

Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.
You are a senior engineer scoping a SAFE refactoring effort. The owner wants critical tech debt cleaned up in files that "grew a lot", with ZERO behavior change and NO quality regression. Propose only refactors that are: (a) behavior-PRESERVING (pure moves/extractions/dedup/dead-code removal ? NOT redesigns, NOT logic changes), (b) backed by EXISTING tests that would catch a regression (assess this honestly per proposal ? if coverage is weak/none, say so and rate risk higher), (c) cleanly separable into ONE reviewable PR. For each: the exact seam (what moves where), the behavior-preservation argument, honest test_coverage, risk, est diff size, value. PREFER high-value + low-risk + well-tested. It is FINE to say a giant function should NOT be refactored yet because coverage is too thin ? flag that as a 'needs tests first' note in debt.

Signals (LOC / longest-functions): gemini.process_video=434L/71br, yolo_hybrid=309L, trajectory_hybrid=231L, motion=204L, compiler.compile_highlights=202L, golden_fixtures=1113L (freeze_real_fixture=186L), main.py=871L (main=168L, _execute_processing=112L), database.py=738L (_init_db=187L), wasb_infer=699L (run=185L), ablation=620L, experiment=579L, tracknet_runner=506L.

CLUSTER: the monster process_video methods ?
backend/pipeline/segmenters/{gemini,yolo_hybrid,trajectory_hybrid,motion}.py. Propose extract-method decompositions into named private helpers; note any shared structure that could move to base.py WITHOUT changing behavior. CRITICAL: assess each segmenter's test coverage (tests/test_*segmenter*, mocked?) ? only recommend refactoring the well-covered ones; flag the rest 'needs characterization tests first'.

2. assistant (2026-06-18T07:54:37.691Z)

I'll scope this refactoring effort. Let me start by reading the target segmenter files and their tests.

3. user (2026-06-18T07:54:39.407Z)

a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)

```
---
__init__.py
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
```

4. user (2026-06-18T07:54:39.763Z)

```

1      from abc import ABC, abstractmethod
2      from typing import List, Dict, Any, Type, Optional
3      from backend.infrastructure.database import Database
4      from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6      class BasePlaySegmenter(ABC):
7          def __init__(self, db: Database, config: Dict[str, Any]):
8              self.db = db
9              self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:

```

```

59         desc = "No description available."
60         res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

5. assistant (2026-06-18T07:54:43.078Z)

The HEAD is a010667, ahead of what was in my env snapshot. Let me read the four target segmenter files and find their test files.

6. user (2026-06-18T07:54:44.037Z)

```

1     import os
2     import time
3     import logging
4     import subprocess
5     import concurrent.futures
6     from typing import List, Dict, Any, Tuple
7     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8     from backend.utils.video import get_video_duration, extract_video_chunk
9     from backend.sports import sport_registry
10    from backend.providers import provider_registry
11    from backend.reporting import run_signals
12
13    logger = logging.getLogger(__name__)
14
15
16    def _chunk_workdir(video_id: str) -> str:
17        """Per-video chunk staging dir under output/chunks/<video_id>.
18
19        Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20        otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21        in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22        """
23        return os.path.join("output", "chunks", video_id)
24
25
26    class GeminiPlaySegmenter(BasePlaySegmenter):
27        @property
28        def name(self) -> str:
29            return "gemini"
30
31        @property
32        def description(self) -> str:
33            return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
34
35        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
36            """
37            Processes a video to detect play segments using Google Gemini Multimodal Video
API,
38            populates SQLite, and indexes detailed events.
39            """
40            # Get sport profile config
41            sport_name = self.config.get("sport", "badminton")
42            try:
43                sport_profile = sport_registry.get(sport_name)
44            except KeyError as e:
45                logger.error(str(e))
46            return []
47
48            # Get AI provider and model config

```

```

49         idx_cfg = self.config.get("indexing", {})
50         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
51         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
52
53         # Get appropriate API key based on the provider
54         api_key_env_var = f"{provider_name.upper()}_API_KEY"
55         api_key = os.environ.get(api_key_env_var)
56         if not api_key:
57             logger.error(
58                 f"{api_key_env_var} environment variable is not set! "
59                 f"To run the {provider_name} segmenter, set your API key. "
60                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
61             )
62             return []
63
64         try:
65             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
66         except KeyError as e:
67             logger.error(str(e))
68             return []
69
70         # A: Check for debug_duration to trim the video first
71         debug_duration = self.config.get("indexing", {}).get("debug_duration")
72         video_to_upload = video_path
73         is_trimmed = False
74         temp_trimmed_path = None
75
76         if debug_duration:
77             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
78             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
79             os.makedirs("output", exist_ok=True)
80             if os.path.exists(temp_trimmed_path):
81                 try:
82                     os.remove(temp_trimmed_path)
83                 except Exception:
84                     pass
85
86             cmd = [
87                 "ffmpeg", "-y",
88                 "-ss", "0",
89                 "-t", str(debug_duration),
90                 "-i", video_path,
91                 "-c", "copy",
92                 temp_trimmed_path
93             ]
94             logger.debug(f"Running command: {' '.join(cmd)}")
95             try:
96                 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
97             logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
98             video_to_upload = temp_trimmed_path
99             is_trimmed = True
100         except Exception as e:
101             logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
102
103
104
105         # Helper to clean up local temp files
106         def cleanup_temp_files():

```

```

107         if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
108             try:
109                 os.remove(temp_trimmed_path)
110             except Exception:
111                 pass
112
113     # 1. Determine actual video duration
114     video_duration = get_video_duration(video_to_upload)
115     if video_duration > 0:
116         logger.info(f"Detected video duration: {video_duration:.1f}s")
117     else:
118         logger.warning("Could not detect video duration. Prompt will not include
duration context.")
119
120     # 2. Decide whether to use chunked processing
121     idx_cfg = self.config.get("indexing", {})
122     chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
123     chunk_overlap = idx_cfg.get("chunk_overlap", 30)      # 30 seconds overlap
124     use_chunking = video_duration > 0 and video_duration > chunk_duration
125     # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
126     # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
127     # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
128     ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
129
130     ai_segments = []
131     # Per-video chunk workdir (concurrency isolation): a concurrent run for another
132     # video keeps its own dir, so cleanup here never removes another run's chunks.
133     chunks_dir = _chunk_workdir(video_id)
134
135     if use_chunking:
136         # --- CHUNKED PROCESSING ---
137         step = chunk_duration - chunk_overlap
138         chunk_starts = []
139         t = 0.0
140         while t < video_duration:
141             chunk_starts.append(t)
142             t += step
143         num_chunks = len(chunk_starts)
144         logger.info(
145             f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
146             f"({chunk_duration}s each, {chunk_overlap}s overlap).")
147         )
148
149         os.makedirs(chunks_dir, exist_ok=True)
150
151         def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
152             chunk_end = min(chunk_start + chunk_duration, video_duration)
153             actual_chunk_dur = chunk_end - chunk_start
154             chunk_label = f"Chunk {ci+1}/{num_chunks}"
155             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
156
157             # Extract chunk with ffmpeg
158             logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
159             success = extract_video_chunk(
160                 video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
161                 reencode=ai_scale_width > 0,
162                 scale_width=ai_scale_width if ai_scale_width > 0 else None,
163             )
164             if not success:
165                 logger.error(f"Failed to extract {chunk_label}. Skipping.")
166                 return chunk_label, [], {}

```

```

167
168         # Measure chunk size
169         try:
170             chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
171         except Exception:
172             chunk_size_mb = 0.0
173
174         # Get prompt from sport profile
175         prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
176
177         # Process this chunk through the provider
178         chunk_segments, metrics = provider.analyze_video(
179             chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
180         )
181         metrics["file_size_mb"] = chunk_size_mb
182         metrics["segments_found"] = len(chunk_segments)
183
184         # Offset timestamps back to full-video coordinates
185         for segment in chunk_segments:
186             if "start_time" in segment:
187                 try:
188                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
189
190                     except (ValueError, TypeError):
191                         pass
192                 if "end_time" in segment:
193                     try:
194                         segment["end_time"] = float(segment["end_time"]) +
chunk_start
195
196                     except (ValueError, TypeError):
197                         pass
198
199                 logger.info(f"[chunk_label] returned {len(chunk_segments)} play
segments.")
200
201         # Clean up chunk file
202         try:
203             os.remove(chunk_path)
204         except Exception:
205             pass
206
207         return chunk_label, chunk_segments, metrics
208
209     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
210     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
211     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
212     # misleading "credits depleted" 429 under 5 simultaneous uploads.
213     all_metrics = {}
214     max_workers = max(1, int(self.config.get("indexing",
{}}).get("ai_max_workers", 5)))
215     logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
216     t_chunking_start = time.time()
217     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
218         futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
219         for future in concurrent.futures.as_completed(futures):
220             label, segments, metrics = future.result()
221             ai_segments.extend(segments)
222             if metrics:
223                 all_metrics[label] = metrics

```

```

222
223         # Oversize refusals must be LOUD: each one is a time range with NO analysis,
224         # so "0 segments / success" here would be a silent failure, not "no
rallies".
225         oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
226         if oversize_skips:
227             logger.error(
228                 f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload
cap and were "
229                 f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time
ranges. "
230                 f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
231                 f"downscaled before upload."
232             )
233             run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
234
235         # Print Summary Log
236         summary_lines = [
237             "",
238             "=" * 60,
239             "                GEMINI CHUNKING SUMMARY",
240             "=" * 60,
241             f" Total time: {time.time() - t_chunking_start:.1f}s",
242             f" Total chunks: {num_chunks}",
243             f" Total play segments found: {len(ai_segments)}",
244             "-" * 60,
245             f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments':<8}",
246             "-" * 60,
247         ]
248         for i in range(num_chunks):
249             lbl = f"Chunk {i+1}/{num_chunks}"
250             m = all_metrics.get(lbl, {})
251             sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
252             upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
253             prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
254             gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
255             seg = m.get('segments_found', 0) if m else 0
256             summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg:<8}")
257         summary_lines.append("=" * 60)
258         logger.info("\n".join(summary_lines))
259
260         # Clean up local temp files
261         cleanup_temp_files()
262         try:
263             os.rmdir(chunks_dir)
264         except Exception:
265             pass
266
267         logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
268
269     else:
270         # --- SINGLE-FILE PROCESSING (short videos) ---
271         try:
272             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
273         except Exception:
274             file_sz = 0.0
275
276         # Get prompt from sport profile
277         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
278         ai_segments, metrics = provider.analyze_video(

```

```

279         video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
280     )
281     if metrics.get("oversize_skipped"):
282         logger.error(
283             f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and
was "
284             f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
285         )
286         run_signals.incr(video_id, "oversize_clips_skipped", 1)
287     summary_lines = [
288         "",
289         "=" * 50,
290         "        GEMINI SINGLE FILE SUMMARY",
291         "=" * 50,
292         f" File Size: {file_sz:.1f} MB",
293         f" Upload time: {metrics.get('upload_time', 0):.1f}s",
294         f" API Processing time: {metrics.get('process_time', 0):.1f}s",
295         f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
296         f" Play segments found: {len(ai_segments)}",
297         "=" * 50,
298     ]
299     logger.info("\n".join(summary_lines))
300     cleanup_temp_files()
301
302     if not ai_segments:
303         return []
304
305     # Populate SQLite DB with parsed play segments
306     segments_data = []
307     logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
308
309     def parse_time(val) -> float:
310         """Converts a timestamp value to total float seconds.
311         Handles float/int pass-through, plain numeric strings, and
312         colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
313         when unexpected formats are encountered."""
314         if isinstance(val, (int, float)):
315             return float(val)
316         if isinstance(val, str):
317             val = val.strip()
318             if ":" in val:
319                 # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
320                 # Convert it, but warn so the issue is visible in logs.
321                 parts = val.split(":")
322                 parts.reverse()
323                 total = 0.0
324                 for i, p in enumerate(parts):
325                     try:
326                         total += float(p) * (60 ** i)
327                     except ValueError:
328                         pass
329                 logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
330                 return total
331             try:
332                 return float(val)
333             except ValueError:
334                 logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
335                 return 0.0
336         logger.warning(f"Unexpected timestamp type {type(val).__name__} for value

```



```

'val}'. Defaulting to 0.0.")
337         return 0.0
338
339     # 7. Parse timestamps and build candidate segment list
340     idx_cfg = self.config.get("indexing", {})
341     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
342     max_segment_duration = 90.0 # Singles play segments almost never exceed this
343
344     candidates = []
345     for idx, segment in enumerate(ai_segments):
346         start = parse_time(segment.get("start_time", 0.0))
347         end = parse_time(segment.get("end_time", 0.0))
348         if start >= end:
349             logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
350             continue
351             candidates.append({
352                 "idx": idx,
353                 "start": start,
354                 "end": end,
355                 "raw": segment
356             })
357
358     # 8. Post-hoc validation pass
359     logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
360     validated = []
361     rejected = []
362
363     for c in candidates:
364         label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
365
366         # A: Reject segments that start beyond the video
367         if video_duration > 0 and c["start"] >= video_duration:
368             rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
369             continue
370
371         # B: Clamp end_time to video duration
372         if video_duration > 0 and c["end"] > video_duration:
373             logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
374             c["end"] = video_duration
375
376         dur = c["end"] - c["start"]
377
378         # C: Reject segments shorter than min_segment_duration
379         if dur < min_segment_duration:
380             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
381             continue
382
383         # D: Flag suspiciously long segments
384         if dur > max_segment_duration:
385             logger.warning(
386                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
387                 f"This may be a detection error. Keeping but flagging."
388             )
389
390         validated.append(c)
391
392     # E: Resolve overlapping segments (keep the longer one)
393     if len(validated) > 1:
394         # Sort by start time

```

```

395         validated.sort(key=lambda r: r["start"])
396         non_overlapping = [validated[0]]
397         for curr in validated[1:]:
398             prev = non_overlapping[-1]
399             # Check if current segment overlaps with the previous one
400             if curr["start"] < prev["end"]:
401                 prev_dur = prev["end"] - prev["start"]
402                 curr_dur = curr["end"] - curr["start"]
403                 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
404                 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
405                 dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
406                 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
407                 if kept == curr:
408                     non_overlapping[-1] = curr # Replace previous with current
409                 else:
410                     non_overlapping.append(curr)
411             validated = non_overlapping
412
413         # Log validation summary
414         if rejected:
415             reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
416             logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
417             logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
418
419         stitch_cfg = self.config.get("stitching", {})
420         begin_pad = stitch_cfg.get("begin_padding", 0.5)
421         end_pad = stitch_cfg.get("end_padding", 0.5)
422
423         total_highlight_dur = 0.0
424         for c in validated:
425             total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
426
427         mins = int(total_highlight_dur // 60)
428         secs = int(total_highlight_dur % 60)
429         logger.info(
430             f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
431             f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
432         )
433
434         # 9. Populate SQLite DB with validated play segments
435         for c in validated:
436             start = c["start"]
437             end = c["end"]
438             segment = c["raw"]
439             dur = end - start
440
441             # Extract shot_count from Gemini or compute estimate if not present
442             shot_count = segment.get("shuttle_exchange_count")
443             if shot_count is None:
444                 shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
445             else:
446                 try:
447                     shot_count = int(shot_count)
448                 except Exception:
449                     shot_count = max(3, int(dur / 0.8))
450
451             # Only store metrics we actually know ? no fabricated data

```

```

452         metadata = {
453             "shot_count": shot_count,
454             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
455             "detector": model_name
456         }
457
458         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
459
460         if segment_id:
461             segments_data.append({
462                 "id": segment_id,
463                 "start_time": start,
464                 "end_time": end,
465                 "duration": dur,
466                 "metadata": metadata
467             })
468
469         return segments_data
470
471     # Register the segmenter
472     segmenter_registry.register(GeminiPlaySegmenter)
473

```

7. user (2026-06-18T07:54:44.531Z)

No files found

8. assistant (2026-06-18T07:54:47.731Z)

Note: `process\_video` here returns a bare list, but base.py declares it returns `Result[...]`.
Let me find the actual test files for these segmenters.

9. user (2026-06-18T07:54:48.301Z)

```

tests\__init__.py
tests\test_ablation.py
tests\test_annotation_providers.py
tests\test_api_endpoints.py
tests\test_async_jobs.py
tests\test_audio_features.py
tests\test_automated_provider.py
tests\test_base.py
tests\test_calibrate_local.py
tests\test_calibrate_wasb.py
tests\test_calibration.py
tests\test_cleanup_caches.py
tests\test_compiler.py
tests\test_config.py
tests\test_config_wasb.py
tests\test_database.py
tests\test_detector_keying.py
tests\test_distill_local.py
tests\test_eval_annotations.py
tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_harness.py
tests\test_eval_metrics.py
tests\test_eval_partial_labels.py
tests\test_eval_regression.py
tests\test_eval_stats.py
tests\test_eval_training.py
tests\test_execution_policy.py

```

tests\test_experiment.py
tests\test_experiment_honest.py
tests\test_export_wasb_segments.py
tests\test_fusion_compare.py
tests\test_fusion_features.py
tests\test_fusion_golden_telemetry.py
tests\test_fusion_hybrid.py
tests\test_gemini.py
tests\test_gemini_refine.py
tests\test_generate_highlights.py
tests\test_geometry.py
tests\test_golden_fixtures.py
tests\test_golden_real_fixtures.py
tests\test_gt_loader.py
tests\test_hashing.py
tests\test_hit_detector.py
tests\test_import_smoke.py
tests\test_job_self_scheduling.py
tests\test_job_worker_loop.py
tests\test_logging_setup.py
tests\test_nightly_regression.py
tests\test_path_safety.py
tests\test_per_user_eval.py
tests\test_per_user_gate.py
tests\test_per_user_regression.py
tests\test_person_detector.py
tests\test_personalization_serving.py
tests\test_promotion.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_rally_gate.py
tests\test_rally_seg_eval.py
tests\test_rally_seq_proto.py
tests\test_rally_slicer.py
tests\test_rally_summary.py
tests\test_reingest_safety.py
tests\test_reliability_hardening.py
tests\test_reporting.py
tests\test_reporting_harvest.py
tests\test_reporting_spec_faq.py
tests\test_results.py
tests\test_run_signals.py
tests\test_run_telemetry.py
tests\test_score_calibration.py
tests\test_segmentation_metrics.py
tests\test_serve_contrast.py
tests\test_served_gate_a_regression.py
tests\test_splits.py
tests\test_sport_profiles.py
tests\test_tolerance_metrics.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_training_gen0.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_upload_download.py
tests\test_validators_base.py
tests\test_variant_tuning.py
tests\test_video.py
tests\test_wasb_cache.py
tests\test_wasb_runner.py
tests\test_watermark.py
tests\test_windowing.py
tests\test_wsl_tilde_quoting.py
tests\test_yolo_hybrid.py

10. user (2026-06-18T07:54:49.085Z)

```
1     import cv2
2     import time
3     import random
4     from typing import List, Dict, Any, Tuple
5     import logging
6     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7     from backend.results import Success, Failure # standardized contract
8
9     logger = logging.getLogger(__name__)
10
11     class MotionPlaySegmenter(BasePlaySegmenter):
12         @property
13         def name(self) -> str:
14             return "motion"
15
16         @property
17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """
25             if not player_pool:
26                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28             cap = cv2.VideoCapture(video_path)
29             if not cap.isOpened():
30                 logger.error(f"Segmenter could not open video: {video_path}")
31                 return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33             fps = cap.get(cv2.CAP_PROP_FPS)
34             total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
35             duration = total_frames / fps if fps > 0 else 0
36
37             # Sub-sample settings (5 FPS for analysis)
38             analysis_fps = 5.0
39             frame_interval = max(1, int(fps / analysis_fps))
40
41             idx_cfg = self.config.get("indexing", {})
42             motion_threshold = idx_cfg.get("motion_threshold", 0.08)
43             min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
44             dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
45
46             prev_gray = None
47             motion_signals = []
48             timestamps = []
49
50             frame_idx = 0
51             processed_count = 0
52             t_start = time.time()
53             # Emit a progress line roughly every 5% of the video so long runs are
54             # observable instead of silent (a full 1080p match is ~tens of minutes).
55             progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
56             next_progress = progress_step
57             logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
58                 f"({~duration/60:.1f} min), analyzing every {frame_interval}th
frame")
59             while True:
60                 # Skip decoding for intermediate frames using cap.grab() for high
```

```

performance
61         for _ in range(frame_interval - 1):
62             if not cap.grab():
63                 break
64             frame_idx += 1
65
66         ret, frame = cap.read()
67         if not ret:
68             break
69
70         # Process frame
71         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
72         # Apply Gaussian Blur to smooth noise
73         gray = cv2.GaussianBlur(gray, (21, 21), 0)
74
75         t = frame_idx / fps
76         timestamps.append(t)
77
78         if prev_gray is not None:
79             # Frame absolute difference (highly optimized motion tracking)
80             diff = cv2.absdiff(prev_gray, gray)
81             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
82
83             # Dilate to fill gaps
84             thresh = cv2.dilate(thresh, None, iterations=2)
85
86             # Motion ratio
87             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
88             motion_signals.append(motion_ratio)
89         else:
90             motion_signals.append(0.0)
91
92         prev_gray = gray
93         frame_idx += 1
94         processed_count += 1
95
96         if progress_step and frame_idx >= next_progress:
97             pct = 100.0 * frame_idx / total_frames
98             elapsed = time.time() - t_start
99             eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
100            logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
101                        f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
102            next_progress += progress_step
103
104            if max_frames is not None and processed_count >= max_frames:
105                logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
106                break
107
108            if frame_idx >= total_frames:
109                break
110
111        cap.release()
112
113        # Smooth signal using a moving average
114        window_size = 5
115        smoothed = []
116        for i in range(len(motion_signals)):
117            start = max(0, i - window_size // 2)
118            end = min(len(motion_signals), i + window_size // 2 + 1)
119            smoothed.append(sum(motion_signals[start:end]) / (end - start))
120

```

```

121     # Detect active play boundaries
122     in_segment = False
123     segment_start = None
124     raw_segments: List[Tuple[float, float]] = []
125
126     for i, t in enumerate(timestamps):
127         signal = smoothed[i]
128         if signal > motion_threshold:
129             if not in_segment:
130                 in_segment = True
131                 segment_start = t
132         else:
133             if in_segment:
134                 in_segment = False
135                 raw_segments.append((segment_start, t))
136
137     # Handle video ending while in active segment
138     if in_segment:
139         raw_segments.append((segment_start, timestamps[-1]))
140
141     # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
142     merged_segments: List[Tuple[float, float]] = []
143     if raw_segments:
144         curr_start, curr_end = raw_segments[0]
145         for next_start, next_end in raw_segments[1:]:
146             if next_start - curr_end < dead_time_merge_gap:
147                 # Merge
148                 curr_end = next_end
149             else:
150                 merged_segments.append((curr_start, curr_end))
151                 curr_start, curr_end = next_start, next_end
152         merged_segments.append((curr_start, curr_end))
153
154     # Post-Processing: 2. Filter segments shorter than min_segment_duration
155     final_segments: List[Tuple[float, float]] = [
156         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
157     ]
158
159     # Populate SQLite DB with segments and metadata events
160     segments_data = []
161     for start, end in final_segments:
162         # Assign Server and Receiver randomly from pool
163         server, receiver = random.sample(player_pool, 2)
164
165         # Formulate dynamic rich tags
166         dur = end - start
167         metadata = {
168             "shot_count": random.randint(4, 25),
169             "intensity": "high" if dur > 12 else "medium",
170             "avg_speed_kmh": round(random.uniform(40, 85), 1)
171         }
172
173         segment_id = self.db.add_play_segment(video_id, start, end, server,
174 receiver, metadata)
175
176         if segment_id:
177             # Add sub-segment events
178             # A: Serve Event (always at start)
179             serve_foul = random.random() < 0.15 # 15% chance of serve foul
180             self.db.add_event(
181                 segment_id=segment_id,
182                 timestamp=start + 0.5,
183                 event_type="serve",
184                 player=server,
185                 details={"status": "foul" if serve_foul else "valid", "type":

```

```

"low_serve" if random.random() > 0.5 else "high_serve"}
185         )
186
187         # B: Smash Events (1 to 3 random smashes per segment)
188         smashes_count = random.randint(0, 3)
189         for _ in range(smashes_count):
190             smash_time = random.uniform(start + 1.5, end - 1.0)
191             smasher = random.choice([server, receiver])
192             self.db.add_event(
193                 segment_id=segment_id,
194                 timestamp=round(smash_time, 2),
195                 event_type="smash",
196                 player=smasher,
197                 details={"speed_kmh": round(random.uniform(120, 250), 1)}
198             )
199
200         # C: Segment Termination Event (foul/winner)
201         foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
202         is_foul = random.random() > 0.4
203         error_player = random.choice([server, receiver])
204
205         self.db.add_event(
206             segment_id=segment_id,
207             timestamp=end - 0.2,
208             event_type="foul" if is_foul else "winner",
209             player=error_player,
210             details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
211         )
212
213         # Store segment representation to return to caller
214         segments_data.append({
215             "id": segment_id,
216             "start_time": start,
217             "end_time": end,
218             "duration": dur,
219             "server": server,
220             "receiver": receiver,
221             "metadata": metadata
222         })
223
224         return Success(value=segments_data)
225
226     # Register the segmenter
227     segmenter_registry.register(MotionPlaySegmenter)
228

```

11. assistant (2026-06-18T07:54:51.112Z)

Now let me read the two hybrid segmenters and their test files.

12. user (2026-06-18T07:54:51.982Z)

```

1     """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2     (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4     Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5     (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6     can be consumed by the future fusion segmenter; this segmenter just orchestrates
7     chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9     Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10    service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11    detection uses torchvision's permissively-licensed (BSD) detectors and the
12    association/tracking that YOLO.track() used to bundle in is provided by

```



```

13     supervision's MIT-licensed ByteTrack.
14
15     The registry key stays "yolo_hybrid" for back-compat with existing configs and the
16     DB `detector` tags, even though no YOLO is involved anymore.
17     """
18     import os
19     import time
20     import logging
21     import concurrent.futures
22     from typing import List, Dict, Any, Tuple
23
24     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
25     from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
26     from backend.utils.video import get_video_duration, extract_video_chunk
27     from backend.sports import sport_registry
28     from backend.providers import provider_registry
29
30     logger = logging.getLogger(__name__)
31
32
33     def _fmt_ts(seconds: float) -> str:
34         """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
35         seconds = max(0, int(seconds))
36         h, rem = divmod(seconds, 3600)
37         m, s = divmod(rem, 60)
38         return f"{h:02d}:{m:02d}:{s:02d}"
39
40
41     class HybridYoloSegmenter(BasePlaySegmenter):
42         def __init__(self, db, config):
43             super().__init__(db, config)
44             # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
45             # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
46             self.person = PersonDetector(config)
47
48             @property
49             def name(self) -> str:
50                 return "yolo_hybrid"
51
52             @property
53             def description(self) -> str:
54                 return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
55                         "candidate filtering, then an AI provider for high-precision semantic
56                         boundaries.")
57
58             def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
59 None, max_frames: int = None) -> List[Dict[str, Any]]:
60                 self.person.lazy_load_model()
61
62                 # Get sport profile config
63                 sport_name = self.config.get("sport", "badminton")
64                 try:
65                     sport_profile = sport_registry.get(sport_name)
66                 except KeyError as e:
67                     logger.error(str(e))
68                     return []
69
70                 # Get AI provider and model config
71                 idx_cfg = self.config.get("indexing", {})
72                 provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
73 "gemini"))
74                 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
75 "gemini-2.5-flash"))

```

```

73         # Get appropriate API key based on the provider
74         api_key_env_var = f"{{provider_name.upper()}}_API_KEY"
75         api_key = os.environ.get(api_key_env_var)
76         if not api_key:
77             logger.error(
78                 f"{{api_key_env_var}} environment variable is not set! "
79                 f"To run the {{provider_name}} segmenter, set your API key. "
80                 f"In PowerShell: $env:{{api_key_env_var}}=\"your_api_key_here\""
81             )
82         return []
83
84     try:
85         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
86     except KeyError as e:
87         logger.error(str(e))
88         return []
89
90     logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{{video_path}}")
91
92     video_duration = get_video_duration(video_path)
93     if video_duration <= 0:
94         logger.error("Invalid video duration.")
95         return []
96
97     chunk_duration = idx_cfg.get("chunk_duration", 300)
98     chunk_overlap = idx_cfg.get("chunk_overlap", 30)
99     velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of
frame width per second
100     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
101     frame_skip = idx_cfg.get("yolo_frame_skip", 3)
102     tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
103     handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
104     ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
105     log_candidates = idx_cfg.get("log_yolo_candidates", True)
106     store_candidates = idx_cfg.get("store_yolo_candidates", True)
107
108     # Snapshot of the params that produced these detections ? stored alongside each
109     # candidate so a fine-tuning run can reproduce / correlate against them.
110     detection_params = {
111         "velocity_thresh": velocity_thresh,
112         "merge_gap": merge_gap,
113         "frame_skip": frame_skip,
114         "tracker": tracker_config,
115         "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
116         "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
117         "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
118     }
119
120     chunks_dir = os.path.join("output", "chunks_yolo")
121     os.makedirs(chunks_dir, exist_ok=True)
122
123     step = chunk_duration - chunk_overlap
124     chunk_starts = []
125     t = 0.0
126     while t < video_duration:
127         chunk_starts.append(t)
128         t += step
129
130     num_chunks = len(chunk_starts)
131     logger.info(f"Video duration: {{video_duration:.1f}}s. Splitting into {{num_chunks}}

```

```

chunks.")
132
133     all_candidate_windows = []
134
135     t_start = time.time()
136     prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)
137
138     if num_chunks > 1 and prefetch_workers > 0:
139         # --- PIPELINED PROCESSING ---
140         # GPU inference must stay sequential (single GPU), but ffmpeg chunk
141         # extraction is pure I/O. We pre-extract upcoming chunks in background
142         # threads so extraction of chunk N+1 overlaps with inference on chunk N.
143         logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")
144
145         extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
146
147         def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
148             chunk_end = min(cs + chunk_duration, video_duration)
149             actual_dur = chunk_end - cs
150             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
151             success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
152             return (ci, cs, chunk_path, success)
153
154         # Submit all extractions upfront ? they'll queue and run as workers free up
155         extract_futures = [
156             extract_executor.submit(_extract_chunk, ci, cs)
157             for ci, cs in enumerate(chunk_starts)
158         ]
159
160         # Process results in order: wait for extraction, then run inference on GPU
161         for future in extract_futures:
162             ci, chunk_start, chunk_path, success = future.result()
163             chunk_end = min(chunk_start + chunk_duration, video_duration)
164             logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
165
166             if not success:
167                 logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
168                 continue
169
170             windows = self.person.extract_action_windows_from_chunk(
171                 chunk_path, chunk_start, velocity_thresh, merge_gap,
172                 frame_skip=frame_skip, chunk_index=ci
173             )
174             logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
175             all_candidate_windows.extend(windows)
176
177             try:
178                 os.remove(chunk_path)
179             except Exception:
180                 pass
181
182             extract_executor.shutdown(wait=False)
183     else:
184         # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
185         for ci, chunk_start in enumerate(chunk_starts):
186             chunk_end = min(chunk_start + chunk_duration, video_duration)
187             actual_dur = chunk_end - chunk_start
188             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
189
190             logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")

```

```

191         success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
192         if not success:
193             continue
194
195         windows = self.person.extract_action_windows_from_chunk(
196             chunk_path, chunk_start, velocity_thresh, merge_gap,
197             frame_skip=frame_skip, chunk_index=ci
198         )
199         logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
200         all_candidate_windows.extend(windows)
201
202         try:
203             os.remove(chunk_path)
204         except Exception:
205             pass
206
207     # Deduplicate overlapping candidate windows across chunks (chunks overlap by
208     # chunk_overlap, so the same rally can surface in two adjacent chunks).
209     def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
210         fa, fb = a["active_frame_count"], b["active_frame_count"]
211         total = fa + fb or 1
212         return {
213             "start": a["start"],
214             "end": max(a["end"], b["end"]),
215             "active_frame_count": fa + fb,
216             "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
217             # Frame-weighted mean so longer sub-windows dominate proportionally.
218             "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
219             "track_id_count": max(a["track_id_count"], b["track_id_count"]),
220             "chunk_index": a["chunk_index"],
221         }
222
223     if all_candidate_windows:
224         all_candidate_windows.sort(key=lambda w: w["start"])
225         merged_windows = [all_candidate_windows[0]]
226         for current in all_candidate_windows[1:]:
227             last = merged_windows[-1]
228             if current["start"] <= last["end"]: # Overlap
229                 merged_windows[-1] = _merge_stats(last, current)
230             else:
231                 merged_windows.append(current)
232         all_candidate_windows = merged_windows
233
234     logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
235     logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
236
237     # Per-rally console log (final, full-video timestamps) for video cross-
verification.
238     if log_candidates:
239         for w in all_candidate_windows:
240             logger.info(
241                 f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
242                 f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}"
243                 f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
244                 f"tracks={w['track_id_count']}"
245             )
246
247     # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
248     # so confirmed AI segments can be linked back in Phase 4.
249     candidate_ids = [None] * len(all_candidate_windows)

```

```

250         if store_candidates:
251             for i, w in enumerate(all_candidate_windows):
252                 stats = {
253                     "peak_velocity": w["peak_velocity"],
254                     "mean_velocity": w["mean_velocity"],
255                     "active_frame_count": w["active_frame_count"],
256                     "track_id_count": w["track_id_count"],
257                 }
258                 # Use peak velocity (capped at 1.0) as a coarse generic confidence
259                 confidence = min(1.0, w["peak_velocity"])
260                 candidate_ids[i] = self.db.add_candidate_segment(
261                     video_id, detector="yolo_hybrid",
262                     start_time=w["start"], end_time=w["end"],
263                     chunk_index=w["chunk_index"], confidence=confidence,
264                     stats=stats, detection_params=detection_params,
265                 )
266
267             try:
268                 os.rmdir(chunks_dir)
269             except Exception:
270                 pass
271
272             # Phase 3: AI Provider Handoff
273             gemini_segments = []
274             gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
275             os.makedirs(gemini_dir, exist_ok=True)
276
277             logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
278 {len(all_candidate_windows)} candidate windows...")
279
280             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
281                 w_start = max(0, window["start"] - handoff_padding)
282                 w_end = min(video_duration, window["end"] + handoff_padding)
283                 w_dur = w_end - w_start
284
285                 w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
286                 # Re-encode short handoff clips to avoid keyframe-boundary drift
287                 success = extract_video_chunk(video_path, w_start, w_dur, w_path,
288 reencode=True)
289
290                 if not success:
291                     return []
292
293                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
294                 segments, metrics = provider.analyze_video(w_path, prompt,
295 chunk_duration=w_dur, label=f"Handoff {wi+1}")
296
297                 # Coordinate mapping ? shift local timestamps back to full-video time
298                 valid_segments = []
299                 for segment in segments:
300                     try:
301                         segment["start_time"] = float(segment["start_time"]) + w_start
302                         segment["end_time"] = float(segment["end_time"]) + w_start
303                         # Stamp the source candidate so Phase 4 can link
304                         segment["candidate_id"] = candidate_ids[wi]
305                         valid_segments.append(segment)
306                     except (ValueError, TypeError, KeyError) as e:
307                         logger.warning(f"Dropping segment with unparseable timestamps:
308 {segment} ? {e}")
309
310                 prediction->confirmation.
311                 try:
312                     os.remove(w_path)
313                 except OSError:
314                     pass

```

```

309
310         return valid_segments
311
312         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
313             futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
314             # Iterate in submission order to preserve temporal ordering
315             for future in futures:
316                 gemini_segments.extend(future.result())
317
318             try:
319                 os.rmdir(gemini_dir)
320             except OSError:
321                 pass
322
323             logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
324
325             # Phase 4: DB Population
326             segments_data = []
327             for segment in gemini_segments:
328                 try:
329                     start = float(segment["start_time"])
330                     end = float(segment["end_time"])
331                     dur = end - start
332
333                     shot_count = segment.get("shuttle_exchange_count")
334                     if shot_count is None:
335                         shot_count = max(3, int(dur / 0.8))
336                     else:
337                         try:
338                             shot_count = int(shot_count)
339                         except (ValueError, TypeError):
340                             shot_count = max(3, int(dur / 0.8))
341
342                     metadata = {
343                         "shot_count": shot_count,
344                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
345                         "detector": f"yolo-hybrid-{model_name}"
346                     }
347
348                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
349                     if segment_id:
350                         # Link back to the candidate that produced this segment. A candidate
351                         # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
352                         # the last one ? the link table records "this candidate was
confirmed".
353                         candidate_id = segment.get("_candidate_id")
354                         if candidate_id is not None:
355                             self.db.link_candidate_to_segment(candidate_id, segment_id)
356                         segments_data.append({
357                             "id": segment_id,
358                             "start_time": start,
359                             "end_time": end,
360                             "duration": dur,
361                             "metadata": metadata
362                         })
363                     except Exception as e:
364                         logger.warning(f"Skipping invalid segment struct: {e}")
365
366             return segments_data

```

```

367
368     # Register the segmenter
369     segmenter_registry.register(HybridYoloSegmenter)
370

```

13. user (2026-06-18T07:54:55.984Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35
36
37    def _fmt_ts(seconds: float) -> str:
38        seconds = max(0, int(seconds))
39        h, rem = divmod(seconds, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44    class TrajectoryHybridSegmenter(BasePlaySegmenter):
45        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46        boundaries."""
47
48        #: config section under `indexing` (velocity_threshold lives there), the
49        #: output subdir for trajectories, and the metadata detector-tag prefix.
50        family: str = ""
51        #: human-readable label used in log lines.
52        display_label: str = ""
53
54        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55        Any]]:
56            """Return (runner, detector-specific detection_params extras)."""
57            raise NotImplementedError

```

```

57     @staticmethod
58     def _probe_fps_width(video_path: str) -> tuple:
59         """Return (fps, frame_width) for a video, with sensible fallbacks."""
60         cap = cv2.VideoCapture(video_path)
61         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63         cap.release()
64         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66     @staticmethod
67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
87         BASE
88         returns exactly the 4 motion keys, so the trajectory twins persist byte-
89         identical
90         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
91         features). ``points`` are the whole-video trajectory points
92         (TrajectoryPoint)."""
93         return {
94             "peak_velocity": cw["peak_velocity"],
95             "mean_velocity": cw["mean_velocity"],
96             "active_frame_count": cw["active_frame_count"],
97             "track_id_count": cw["track_id_count"],
98         }
99
100     def _select_for_handoff(self, windows: List[Dict[str, Any]],
101                            window_stats: List[Dict[str, Any]],
102                            idx_cfg: Dict[str, Any]) -> List[int]:
103         """Indices of the candidate windows that proceed to the AI handoff (and thus may
104         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
105         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
106         Persisted candidates are NOT affected ? they remain the full superset for
107         offline
108         re-scoring."""
109         return list(range(len(windows)))
110
111     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
112                      player_pool: Optional[List[str]] = None,
113                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
114         # override-ignore: the ABC declares the Result contract (Success|Failure)
115         # but every live segmenter still returns a bare list ? the documented
116         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
117         label = self.display_label
118         # --- Sport profile + AI provider wiring (identical across segmenters) ---
119         sport_name = self.config.get("sport", "badminton")
120         try:
121             sport_profile = sport_registry.get(sport_name)

```



```

117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
123         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
124         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
125         # measure it against the Gemini path, or to avoid the cloud entirely). With
126         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
127         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
128         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
129         if skip_ai:
130             model_name = "local-noai"
131             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
132                         "be emitted as rallies; no %s handoff, no API key needed.",
133                         label, provider_name)
134         else:
135             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
136             api_key_env_var = f"{provider_name.upper()}_API_KEY"
137             api_key = os.environ.get(api_key_env_var)
138             if not api_key:
139                 logger.error(
140                     f"{api_key_env_var} environment variable is not set! "
141                     f"To run the {provider_name} handoff, set your API key. "
142                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
143                 )
144             return []
145         try:
146             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
147         except KeyError as e:
148             logger.error(str(e))
149             return []
150
151         video_duration = get_video_duration(video_path)
152         if video_duration <= 0:
153             logger.error("Invalid video duration.")
154             return []
155         fps, frame_width = self._probe_fps_width(video_path)
156
157         # --- Runner config + windowing thresholds ---
158         runner, runner_params = self._build_runner(idx_cfg)
159
160         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
163         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
164         max_window_duration = idx_cfg.get("max_window_duration", 0.0)
165         window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
166         window_hard_cap = idx_cfg.get("window_hard_cap", False)
167         window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
168         inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
169         inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
170         window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
171         window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
172         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)

```

```

173         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
174         log_candidates = idx_cfg.get("log_yolo_candidates", True)
175         store_candidates = idx_cfg.get("store_yolo_candidates", True)
176
177         detection_params = {
178             "detector": self.name,
179             "velocity_thresh": velocity_thresh,
180             "merge_gap": merge_gap,
181             "min_action_window_duration": min_window_duration,
182             **runner_params,
183         }
184
185         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
186         ok, msg = runner.healthcheck()
187         if not ok:
188             logger.error("%s WSL environment not ready: %s", label, msg)
189             return []
190         logger.info("%s WSL env OK: %s", label, msg)
191
192         # --- Phase 2: trajectory -> candidate rally windows ---
193         # Trajectory detectors process the whole video in one pass (they carry
194         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
195         t_start = time.time()
196         out_dir = os.path.join("output", self.family)
197         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
198         if not csv_path:
199             logger.error("%s produced no trajectory; aborting.", label)
200             return []
201
202         points = runner.parse_trajectory_csv(csv_path)
203         logger.info("Parsed %d trajectory points (%d visible).",
204                     len(points), sum(1 for p in points if p.visible))
205
206         all_candidate_windows = runner.trajectory_to_action_windows(
207             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
208             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
209             min_window_duration=min_window_duration, chunk_index=0,
210             max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
211             window_hard_cap=window_hard_cap,
212             window_inactivity_end=window_inactivity_end,
213             inactivity_rally_thresh=inactivity_rally_thresh,
214             inactivity_min_density=inactivity_min_density,
215             window_blob_fallback=window_blob_fallback,
216             window_blob_factor=window_blob_factor,
217         )
218         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
219                     label, time.time() - t_start, len(all_candidate_windows))
220
221         if log_candidates:
222             for cw in all_candidate_windows:
223                 logger.info(f"[{label}] rally]
224                 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
225                     f"({cw['end'] - cw['start']:.1f}s) |
226                 frames={cw['active_frame_count']} "
227                     f"peak_v={cw['peak_velocity']:.3f}
228                 mean_v={cw['mean_velocity']:.3f}")
229
230         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
231         fusion
232         # segmenter adds net_crossings + person-cue features. Computed once, reused for
233         both
234         # persistence and the handoff gate below.
235         window_stats = [
236             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
237             idx_cfg)

```

```

232         for cw in all_candidate_windows
233     ]
234
235     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
236     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
237     if store_candidates:
238         for i, cw in enumerate(all_candidate_windows):
239             candidate_ids[i] = self.db.add_candidate_segment(
240                 video_id, detector=self.name,
241                 start_time=cw["start"], end_time=cw["end"],
242                 chunk_index=cw["chunk_index"], confidence=min(1.0,
243                 stats=window_stats[i], detection_params=detection_params,
244                 )
245
246     if not all_candidate_windows:
247         return []
248
249     # --- Phase 3: AI provider handoff over padded candidate clips ---
250     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
251     # candidates above stay the full superset ? only the served play_segments are
252     # gated.
253     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
254     idx_cfg)
255
256     ai_segments: List[dict] = []
257     if skip_ai:
258         # Fully-local: the selected candidate windows ARE the rallies ? emit them
259         verbatim
260         # (no clip extraction, no upload). Boundaries are the local detector's; shot
261         count
262         # falls back to the duration-based estimate (no AI exchange count).
263         ai_segments = [
264             {
265                 "start_time": all_candidate_windows[wi]["start"],
266                 "end_time": all_candidate_windows[wi]["end"],
267                 "shuttle_exchange_count": None,
268                 "shot_count_confidence": "LocalNoAI",
269                 "_candidate_id": candidate_ids[wi],
270             }
271             for wi in handoff_indices
272         ]
273     logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
274     rallies "
275                 "(no AI handoff).", len(ai_segments))
276
277     else:
278         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
279         os.makedirs(handoff_dir, exist_ok=True)
280         logger.info("Phase 3: %s validation on %d candidate windows...",
281                 provider_name, len(handoff_indices))
282
283         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
284             w_start = max(0, window["start"] - handoff_padding)
285             w_end = min(video_duration, window["end"] + handoff_padding)
286             w_dur = w_end - w_start
287             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
288             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
289             reencode=True):
290                 return []
291             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
292             segments, _ = provider.analyze_video(w_path, prompt,
293             chunk_duration=w_dur,
294                 label=f"Handoff {wi+1}")
295             valid = []

```

```

288         for seg in segments:
289             try:
290                 seg["start_time"] = float(seg["start_time"]) + w_start
291                 seg["end_time"] = float(seg["end_time"]) + w_start
292                 seg["_candidate_id"] = candidate_ids[wi]
293                 valid.append(seg)
294             except (ValueError, TypeError, KeyError) as e:
295                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
296         try:
297             os.remove(w_path)
298         except OSError:
299             pass
300         return valid
301
302     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
303         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
304                     for wi in handoff_indices]
305         for future in futures:
306             ai_segments.extend(future.result())
307         try:
308             os.rmdir(handoff_dir)
309         except OSError:
310             pass
311         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
312
313     # --- Phase 4: DB population + candidate linking ---
314     segments_data = []
315     for segment in ai_segments:
316         try:
317             start = float(segment["start_time"])
318             end = float(segment["end_time"])
319             dur = end - start
320             metadata = {
321                 "shot_count": self._shot_count_from(segment, dur),
322                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
323                 "detector": f"{self.family}-hybrid-{model_name}",
324             }
325             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
326             if segment_id:
327                 candidate_id = segment.get("_candidate_id")
328                 if candidate_id is not None:
329                     self.db.link_candidate_to_segment(candidate_id, segment_id)
330                 segments_data.append({
331                     "id": segment_id, "start_time": start, "end_time": end,
332                     "duration": dur, "metadata": metadata,
333                 })
334         except Exception as e:
335             logger.warning("Skipping invalid segment struct: %s", e)
336
337     return segments_data
338

```

14. assistant (2026-06-18T07:54:58.411Z)

The trajectory_hybrid is already a well-factored base class with hooks. Now let me read the test files for these four segmenters.

15. user (2026-06-18T07:54:59.838Z)

```

1     import logging
2     import os
3
4     from unittest.mock import MagicMock, patch
5     from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter, _chunk_workdir
6     from backend.reporting import run_signals
7
8
9     def test_chunk_workdir_is_per_video():
10         """Concurrency isolation: two videos must get DIFFERENT chunk workdirs under
11         output/chunks, so one run finishing (and rmdir-ing its dir) can't undo another
12         run's in-flight chunks. Regression for the shared-output/chunks clobber."""
13         a, b = _chunk_workdir("vidA"), _chunk_workdir("vidB")
14         assert a != b
15         assert a == os.path.join("output", "chunks", "vidA")
16         assert os.path.dirname(a) == os.path.dirname(b) == os.path.join("output", "chunks")
17
18         @patch("backend.pipeline.segmenters.gemini.provider_registry")
19         @patch("backend.pipeline.segmenters.gemini.get_video_duration")
20         @patch("os.environ.get")
21         def test_gemini_segmenter_single_file(mock_env_get, mock_get_dur,
22 mock_provider_registry):
23             mock_env_get.return_value = "dummy_api_key"
24             mock_get_dur.return_value = 100.0 # 100s video
25
26             mock_provider = MagicMock()
27             mock_provider.analyze_video.return_value = ([
28                 {"start_time": 10.0, "end_time": 20.0, "shuttle_exchange_count": 5}
29             ], {"upload_time": 1, "process_time": 1, "gen_time": 1})
30             mock_provider_registry.get.return_value = mock_provider
31
32             db_mock = MagicMock()
33             db_mock.add_play_segment.return_value = "segment_1"
34
35             config = {
36                 "indexing": {
37                     "chunk_duration": 300, # Large chunk duration, falls back to single file
38                     "chunk_overlap": 30
39                 }
40             }
41
42             segmenter = GeminiPlaySegmenter(db_mock, config)
43             res = segmenter.process_video("v1", "dummy.mp4")
44
45             assert len(res) == 1
46             assert res[0]["id"] == "segment_1"
47             assert res[0]["start_time"] == 10.0
48             assert res[0]["end_time"] == 20.0
49             assert res[0]["metadata"]["shot_count"] == 5
50
51             @patch("backend.pipeline.segmenters.gemini.provider_registry")
52             @patch("backend.pipeline.segmenters.gemini.get_video_duration")
53             @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
54             @patch("os.environ.get")
55             def test_gemini_segmenter_chunked(mock_env_get, mock_extract, mock_get_dur,
56 mock_provider_registry):
57                 mock_env_get.return_value = "dummy_api_key"
58                 mock_get_dur.return_value = 200.0 # 200s video
59                 mock_extract.return_value = True
60
61                 mock_provider = MagicMock()
62                 # Chunk 1 returns 1 segment, Chunk 2 returns another
63                 def analyze_video_side_effect(path, prompt, chunk_duration, label, **kwargs):
64                     if "Chunk 1" in label:
65                         return [{"start_time": 10.0, "end_time": 20.0}], {}

```

```

64         else:
65             return [{"start_time": 30.0, "end_time": 40.0}], {}
66
67     mock_provider.analyze_video.side_effect = analyze_video_side_effect
68     mock_provider_registry.get.return_value = mock_provider
69
70     db_mock = MagicMock()
71     db_mock.add_play_segment.side_effect = ["segment_1", "segment_2"]
72
73     config = {
74         "indexing": {
75             "chunk_duration": 150, # Smaller than video, causes chunking
76             "chunk_overlap": 10
77         }
78     }
79
80     segmenter = GeminiPlaySegmenter(db_mock, config)
81     res = segmenter.process_video("v1", "dummy.mp4")
82
83     # Should be 2 segments
84     assert len(res) == 2
85
86     # 1st chunk start is 0
87     assert res[0]["start_time"] == 10.0
88     # 2nd chunk start is 150-10 = 140
89     # So its local time 30 -> 140+30 = 170.0
90     # Let's just check length, exact timing requires checking array order (which may
vary due to concurrent futures)
91     times = [r["start_time"] for r in res]
92     assert 10.0 in times
93     assert 170.0 in times
94
95
96     def _run_chunked(mock_extract, mock_provider_registry, indexing_overrides=None,
97                     analyze_return=None, video_id="v1"):
98         """Drive a 2-chunk run with everything mocked; returns the extract mock for arg
asserts."""
99         mock_extract.return_value = True
100         mock_provider = MagicMock()
101         mock_provider.analyze_video.return_value = analyze_return or (
102             [{"start_time": 10.0, "end_time": 20.0}], {"upload_time": 0, "process_time": 0,
"gen_time": 0})
103         mock_provider_registry.get.return_value = mock_provider
104         db_mock = MagicMock()
105         db_mock.add_play_segment.return_value = "segment_1"
106         config = {"indexing": {"chunk_duration": 150, "chunk_overlap": 10,
**(indexing_overrides or {})}}
107         segmenter = GeminiPlaySegmenter(db_mock, config)
108         return segmenter.process_video(video_id, "dummy.mp4")
109
110
111     @patch("backend.pipeline.segmenters.gemini.provider_registry")
112     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
113     @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
114     @patch("os.environ.get")
115     def test_chunks_are_reencoded_and_downscaled_by_default(mock_env_get, mock_extract,
mock_get_dur,
116                                                         mock_provider_registry):
117         """A 4K source's stream-copied 90s chunks exceed the provider's upload cap and get
118         refused wholesale ? so AI-handoff chunks must be re-encoded at ai_chunk_scale_width
119         (default 1280) rather than stream-copied. Also fixes keyframe drift at
boundaries."""
120         mock_env_get.return_value = "dummy_api_key"
121         mock_get_dur.return_value = 200.0
122         _run_chunked(mock_extract, mock_provider_registry)

```

```

123     assert mock_extract.call_count == 2
124     for call in mock_extract.call_args_list:
125         assert call.kwargs["reencode"] is True
126         assert call.kwargs["scale_width"] == 1280
127
128
129     @patch("backend.pipeline.segmenters.gemini.provider_registry")
130     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
131     @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
132     @patch("os.environ.get")
133     def test_chunk_scale_width_is_configurable_and_zero_disables(mock_env_get, mock_extract,
mock_get_dur,
134                                                                 mock_provider_registry):
135         mock_env_get.return_value = "dummy_api_key"
136         mock_get_dur.return_value = 200.0
137         # Custom width flows through
138         _run_chunked(mock_extract, mock_provider_registry,
indexing_overrides={"ai_chunk_scale_width": 960})
139         assert mock_extract.call_args.kwargs["scale_width"] == 960
140         assert mock_extract.call_args.kwargs["reencode"] is True
141         # 0 = legacy stream-copy (no re-encode, no scaling)
142         mock_extract.reset_mock()
143         _run_chunked(mock_extract, mock_provider_registry,
indexing_overrides={"ai_chunk_scale_width": 0})
144         assert mock_extract.call_args.kwargs["reencode"] is False
145         assert mock_extract.call_args.kwargs["scale_width"] is None
146
147
148     @patch("backend.pipeline.segmenters.gemini.provider_registry")
149     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
150     @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
151     @patch("os.environ.get")
152     def test_oversize_chunk_skips_are_loud_and_reach_run_signals(mock_env_get, mock_extract,
mock_get_dur,
153                                                                 mock_provider_registry,
caplog):
154         """When the provider refuses chunks as oversize, the segmenter must log an ERROR and
155         record the count in run_signals so the per-video report can flag it ? never a silent
156         '0 segments / success'."""
157         mock_env_get.return_value = "dummy_api_key"
158         mock_get_dur.return_value = 200.0
159         run_signals.pop("vid_oversize") # isolate from any prior state
160         with caplog.at_level(logging.ERROR, logger="backend.pipeline.segmenters.gemini"):
161             res = _run_chunked(
162                 mock_extract, mock_provider_registry, video_id="vid_oversize",
163                 analyze_return=([], {"upload_time": 0, "process_time": 0, "gen_time": 0,
164                                     "oversize_skipped": True}),
165             )
166         assert res == []
167         assert run_signals.pop("vid_oversize") == {"oversize_clips_skipped": 2}
168         assert any("SKIPPED WITHOUT ANALYSIS" in r.message for r in caplog.records)
169
170
171     @patch("backend.pipeline.segmenters.gemini.provider_registry")
172     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
173     @patch("os.environ.get")
174     def test_oversize_single_file_skip_reaches_run_signals(mock_env_get, mock_get_dur,
mock_provider_registry, caplog):
175         mock_env_get.return_value = "dummy_api_key"
176         mock_get_dur.return_value = 100.0 # < chunk_duration -> single-file path
177         mock_provider = MagicMock()
178         mock_provider.analyze_video.return_value = (
179             [], {"upload_time": 0, "process_time": 0, "gen_time": 0, "oversize_skipped":
True})
180         mock_provider_registry.get.return_value = mock_provider

```

```

181     run_signals.pop("vid_single_oversize")
182     config = {"indexing": {"chunk_duration": 300, "chunk_overlap": 30}}
183     segmenter = GeminiPlaySegmenter(MagicMock(), config)
184     with caplog.at_level(logging.ERROR, logger="backend.pipeline.segmenters.gemini"):
185         res = segmenter.process_video("vid_single_oversize", "dummy.mp4")
186         assert res == []
187         assert run_signals.pop("vid_single_oversize") == {"oversize_clips_skipped": 1}
188         assert any("SKIPPED WITHOUT ANALYSIS" in r.message for r in caplog.records)
189

```

16. user (2026-06-18T07:55:00.056Z)

```

1     """Orchestration tests for HybridYoloSegmenter.
2
3     Stage-1 detection now lives in `PersonDetector` (see test_person_detector.py); these
4     tests mock the segmenter's `self.person` cue seams and assert the segmenter's
5     process_video OUTPUT is unchanged ? that is the regression guard for the fusion-P1
6     extraction (the detector moved, behaviour did not). cv2.VideoCapture is patched in the
7     `person_detector` module now (that is where the capture loop runs).
8     """
9     from unittest.mock import MagicMock, patch
10    from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
11
12
13    def _make_cap_mock(num_frames=300):
14        """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
15        import cv2
16        cap_mock = MagicMock()
17        cap_mock.isOpened.return_value = True
18
19        def mock_get_prop(prop):
20            if prop == cv2.CAP_PROP_FPS:
21                return 30.0
22            if prop == cv2.CAP_PROP_FRAME_WIDTH:
23                return 1000.0
24            return 0.0
25
26        cap_mock.get.side_effect = mock_get_prop
27        cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
28        return cap_mock
29
30
31    def _make_detections(num_frames=300, step=5, track_id=1):
32        """Per-frame return values for a mocked detect_and_track: one steadily-moving
33        person. Each entry is the list of (track_id, bbox) for that frame, where the box
34        advances `step` px/frame so its normalised velocity clears the test thresholds.
35        """
36        out = []
37        for i in range(num_frames):
38            x = i * step
39            out.append([(track_id, (float(x), 0.0, float(x + 10), 10.0))])
40        return out
41
42
43    def _mock_stage1(segmenter, num_frames=300, step=5):
44        """Wire the segmenter's person-cue detection+tracking to deterministic fakes so
45        tests never need torch model inference, GPU, or a real supervision tracker."""
46        segmenter.person.model = MagicMock() # short-circuits
47        lazy_load_model
48        segmenter.person.make_tracker = MagicMock(return_value=MagicMock())
49        segmenter.person.detect_and_track =
50        MagicMock(side_effect=_make_detections(num_frames, step))
51
52    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")

```



```

52 @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
53 @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
54 @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
55 @patch("os.environ.get")
56 def test_yolo_hybrid_segmenter(mock_env_get, mock_provider_registry, mock_extract,
57                               mock_get_dur, mock_cap):
58     mock_env_get.return_value = "dummy_api_key"
59     mock_get_dur.return_value = 10.0
60     mock_extract.return_value = True
61
62     # Mock provider returned play segment
63     mock_provider = MagicMock()
64     mock_provider.analyze_video.return_value = ([
65         {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
66     ], {})
67     mock_provider_registry.get.return_value = mock_provider
68
69     db_mock = MagicMock()
70     db_mock.add_play_segment.return_value = "yolo_segment_1"
71
72     config = {
73         "indexing": {
74             "use_gpu": False,
75             "chunk_duration": 100,
76             "chunk_overlap": 10,
77             "yolo_velocity_threshold": 0.1,
78             "dead_time_merge_gap": 2.0,
79             # Process every frame so all 300 frames are tracked
80             "yolo_frame_skip": 1,
81             "min_action_window_duration": 1.0,
82         }
83     }
84
85     segmenter = HybridYoloSegmenter(db_mock, config)
86     _mock_stage1(segmenter)
87     mock_cap.return_value = _make_cap_mock()
88
89     res = segmenter.process_video("v1", "dummy.mp4")
90
91     assert len(res) == 1
92     assert res[0]["id"] == "yolo_segment_1"
93     assert res[0]["start_time"] == 0.0
94     assert res[0]["end_time"] == 3.0
95
96
97 @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
98 @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
99 @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
100 @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
101 @patch("os.environ.get")
102 def test_yolo_hybrid_no_api_key(mock_env_get, mock_provider_registry, mock_extract,
103                                mock_get_dur, mock_cap):
104     """Segmenter should return empty when GEMINI_API_KEY is missing."""
105     mock_env_get.return_value = None
106
107     db_mock = MagicMock()
108     config = {"indexing": {"use_gpu": False}}
109
110     segmenter = HybridYoloSegmenter(db_mock, config)
111     segmenter.person.model = MagicMock() # short-circuit lazy model load
112
113     res = segmenter.process_video("v1", "dummy.mp4")
114     assert res == []
115
116

```

```

117     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
118     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
119     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
120     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
121     @patch("os.environ.get")
122     def test_yolo_hybrid_drops_bad_timestamps(mock_env_get, mock_provider_registry,
mock_extract,
123                                             mock_get_dur, mock_cap):
124         """Play segments with unparseable timestamps should be dropped, not silently
kept."""
125         mock_env_get.return_value = "dummy_api_key"
126         mock_get_dur.return_value = 10.0
127         mock_extract.return_value = True
128
129         # Provider returns one good segment and one with garbage timestamps
130         mock_provider = MagicMock()
131         mock_provider.analyze_video.return_value = ([
132             {"start_time": 1.0, "end_time": 3.0},
133             {"start_time": "N/A", "end_time": "??"},
134         ], {})
135         mock_provider_registry.get.return_value = mock_provider
136
137         db_mock = MagicMock()
138         db_mock.add_play_segment.return_value = "segment_ok"
139
140         config = {
141             "indexing": {
142                 "use_gpu": False,
143                 "chunk_duration": 100,
144                 "chunk_overlap": 10,
145                 "yolo_velocity_threshold": 0.05,
146                 "dead_time_merge_gap": 2.0,
147                 "yolo_frame_skip": 1,
148                 "min_action_window_duration": 0.5,
149             }
150         }
151
152         segmenter = HybridYoloSegmenter(db_mock, config)
153         _mock_stage1(segmenter)
154         mock_cap.return_value = _make_cap_mock()
155
156         res = segmenter.process_video("v1", "dummy.mp4")
157
158         # Only the valid segment should survive ? the "N/A" one should be dropped
159         assert len(res) == 1
160         assert res[0]["start_time"] == 1.0
161
162
163     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
164     def test_extract_windows_returns_enriched_dicts(mock_cap):
165         """The person cue's window extraction should return dicts with motion stats."""
166         config = {"indexing": {"min_action_window_duration": 1.0}}
167         segmenter = HybridYoloSegmenter(MagicMock(), config)
168         _mock_stage1(segmenter)
169         mock_cap.return_value = _make_cap_mock()
170
171         windows = segmenter.person.extract_action_windows_from_chunk(
172             "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
173             frame_skip=1, chunk_index=2,
174         )
175
176         assert len(windows) >= 1
177         w = windows[0]
178         assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
179                             "mean_velocity", "track_id_count", "chunk_index"}

```

```

180     assert w["chunk_index"] == 2
181     assert w["start"] >= 5.0 # offset into full-video coordinates
182     assert w["active_frame_count"] > 0
183     assert w["peak_velocity"] >= w["mean_velocity"] > 0
184     assert w["track_id_count"] == 1
185
186
187     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
188     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
189     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
190     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
191     @patch("os.environ.get")
192     def test_yolo_hybrid_persists_and_links_candidates(mock_env_get, mock_provider_registry,
193                                                       mock_extract, mock_get_dur,
194                                                       mock_cap):
195         """Raw candidates should be stored, and confirmed ones linked to their segment."""
196         mock_env_get.return_value = "dummy_api_key"
197         mock_get_dur.return_value = 10.0
198         mock_extract.return_value = True
199
200         mock_provider = MagicMock()
201         mock_provider.analyze_video.return_value = ([
202             {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
203         ], {})
204         mock_provider_registry.get.return_value = mock_provider
205
206         db_mock = MagicMock()
207         db_mock.add_candidate_segment.return_value = 77
208         db_mock.add_play_segment.return_value = 101
209
210         config = {
211             "indexing": {
212                 "use_gpu": False,
213                 "chunk_duration": 100,
214                 "chunk_overlap": 10,
215                 "yolo_velocity_threshold": 0.05,
216                 "dead_time_merge_gap": 2.0,
217                 "yolo_frame_skip": 1,
218                 "min_action_window_duration": 1.0,
219             }
220         }
221
222         segmenter = HybridYoloSegmenter(db_mock, config)
223         _mock_stage1(segmenter)
224         mock_cap.return_value = _make_cap_mock()
225
226         res = segmenter.process_video("v1", "dummy.mp4")
227
228         assert len(res) == 1
229         # Candidate persisted with the right detector + reproducibility params
230         assert db_mock.add_candidate_segment.called
231         _, kwargs = db_mock.add_candidate_segment.call_args
232         assert kwargs["detector"] == "yolo_hybrid"
233         assert "peak_velocity" in kwargs["stats"]
234         assert kwargs["detection_params"]["frame_skip"] == 1
235         # Confirmed segment linked back to its candidate
236         db_mock.link_candidate_to_segment.assert_called_once_with(77, 101)
237
238     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
239     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
240     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
241     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
242     @patch("os.environ.get")
243     def test_yolo_hybrid_store_candidates_disabled(mock_env_get, mock_provider_registry,

```

```

244                                     mock_extract, mock_get_dur, mock_cap):
245     """store_yolo_candidates=False should skip DB persistence (and linking)."""
246     mock_env_get.return_value = "dummy_api_key"
247     mock_get_dur.return_value = 10.0
248     mock_extract.return_value = True
249
250     mock_provider = MagicMock()
251     mock_provider.analyze_video.return_value = ([
252         {"start_time": 0.0, "end_time": 3.0}
253     ], {})
254     mock_provider_registry.get.return_value = mock_provider
255
256     db_mock = MagicMock()
257     db_mock.add_play_segment.return_value = 101
258
259     config = {
260         "indexing": {
261             "use_gpu": False,
262             "chunk_duration": 100,
263             "chunk_overlap": 10,
264             "yolo_velocity_threshold": 0.05,
265             "dead_time_merge_gap": 2.0,
266             "yolo_frame_skip": 1,
267             "min_action_window_duration": 1.0,
268             "store_yolo_candidates": False,
269         }
270     }
271
272     segmenter = HybridYoloSegmenter(db_mock, config)
273     _mock_stage1(segmenter)
274     mock_cap.return_value = _make_cap_mock()
275
276     segmenter.process_video("v1", "dummy.mp4")
277
278     db_mock.add_candidate_segment.assert_not_called()
279     db_mock.link_candidate_to_segment.assert_not_called()
280

```

17. assistant (2026-06-18T07:55:03.381Z)

Now let me check for motion segmenter tests and the trajectory_hybrid equivalence test.

18. user (2026-06-18T07:55:04.974Z)

```

Found 6 files
mypy.ini
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\CODE_MAP.md
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\__init__.py
.coveragerc

```

19. user (2026-06-18T07:55:05.671Z)

```

1     """Equivalence contract for the trajectory-hybrid twins.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins that drifted (the
4     shot_count parse); both are now thin subclasses of TrajectoryHybridSegmenter.
5     These tests run BOTH through the same scenarios and assert identical P3/P4
6     behavior, so any future divergence has to be introduced deliberately, in the
7     base, for both at once.
8     """
9     from unittest.mock import MagicMock, patch
10

```

```

11     import pytest
12
13     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
14     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
15     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
16
17     TWINS = [
18         (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
19         (WasbHybridSegmenter, "wasb", "wasb_hybrid"),
20     ]
21
22
23     def _window(start=1.0, end=4.0):
24         return {
25             "start": start, "end": end, "active_frame_count": 30,
26             "peak_velocity": 0.4, "mean_velocity": 0.2, "track_id_count": 1,
27             "chunk_index": 0,
28         }
29
30
31     def _runner(windows=None):
32         runner = MagicMock()
33         runner.healthcheck.return_value = (True, "env ok")
34         runner.run_predict.return_value = "out.csv"
35         runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
36         runner.trajectory_to_action_windows.return_value = windows if windows is not None
37     else [_window()]
38     return runner
39
40     def _run(cls, provider_segments, db=None):
41         """Run one segmenter class through the standard mocked pipeline."""
42         provider = MagicMock()
43         provider.analyze_video.return_value = (provider_segments, {})
44         db = db or MagicMock()
45         db.add_candidate_segment.return_value = 55
46         db.add_play_segment.return_value = 200
47
48         with patch.object(cls, "_build_runner", return_value=(_runner(), {"weights_path":
49 "w"})), \
50             patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
52 return_value=30.0), \
53             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
54 return_value=True), \
55             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
56 preg, \
57             patch("os.environ.get", return_value="dummy_key"):
58             preg.get.return_value = provider
59             seg = cls(db, {"indexing": {}})
60             return db, seg.process_video("v1", "clip.mp4")
61
62 @pytest.mark.parametrize("cls,family,detector", TWINS)
63 def test_detector_identity_flows_into_db_rows(cls, family, detector):
64     db, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count":
65 4}])
66     assert len(res) == 1
67     _, kwargs = db.add_candidate_segment.call_args
68     assert kwargs["detector"] == detector
69     assert kwargs["detection_params"]["detector"] == detector
70     # P4 metadata tags the family + model.
71     assert res[0]["metadata"]["detector"].startswith(f"{family}-hybrid-")
72     db.link_candidate_to_segment.assert_called_once_with(55, 200)

```

```

70
71 @pytest.mark.parametrize("cls,family,detector", TWINS)
72 @pytest.mark.parametrize("exchange,expected", [
73     (4, 4),          # provider-reported count wins
74     ("7", 7),        # numeric strings coerce
75     (None, 3),        # explicit null -> duration estimate (2.5s/0.8 -> 3 floor)
76     ("garbage", 3),   # unparseable -> same estimate
77 ])
78 def test_shot_count_behavior_is_identical(cls, family, detector, exchange, expected):
79     """The historical drift point: wasb_hybrid relied on int(None) raising into
80     the fallback while tracknet_hybrid short-circuited. One implementation now."""
81     seg_dict = {"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": exchange}
82     _, res = _run(cls, [seg_dict])
83     assert res[0]["metadata"]["shot_count"] == expected
84
85
86 @pytest.mark.parametrize("cls,family,detector", TWINS)
87 def test_provider_offsets_applied_identically(cls, family, detector):
88     # Window [1,4] + 2.0s default padding -> clip starts at max(0, -1) = 0.0;
89     # provider-relative 0.5 stays 0.5 absolute here.
90     _, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
91     assert res[0]["start_time"] == 0.5
92     assert res[0]["end_time"] == 3.0
93
94
95 def test_base_is_not_registered():
96     from backend.pipeline.segmenters.base import segmenter_registry
97     available = segmenter_registry.list_available()
98     assert "tracknet_hybrid" in available
99     assert "wasb_hybrid" in available
100     # The base engine itself must not appear as a selectable segmenter.
101     assert not any(isinstance(cls, type) and cls is TrajectoryHybridSegmenter
102                     for cls in available.values())
103
104
105 @pytest.mark.parametrize("cls,family,detector", TWINS)
106 def test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
107     """The fusion-P2 enrichment seam must leave the twins' persisted stats byte-
108 identical:
109 the base _enrich_candidate_stats is identity, so NO net_crossings / person keys leak
110 in."""
111     db, _res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
112     _, kwargs = db.add_candidate_segment.call_args
113     assert set(kwargs["stats"]) == {
114         "peak_velocity", "mean_velocity", "active_frame_count", "track_id_count"}
115
116 @pytest.mark.parametrize("cls,family,detector", TWINS)
117 def test_base_fusion_hooks_are_identity(cls, family, detector):
118     """Directly pin the two fusion seams on the base: stats = the 4 motion keys
119 verbatim,
120 and every window is selected for handoff (no gating)."""
121     seg = cls(MagicMock(), {"indexing": {}})
122     cw = _window()
123     stats = seg._enrich_candidate_stats(cw, [], 30.0, 1280.0, "v.mp4", {})
124     assert stats == {"peak_velocity": 0.4, "mean_velocity": 0.2,
125                     "active_frame_count": 30, "track_id_count": 1}
126     assert seg._select_for_handoff([cw, cw, cw], [stats, stats, stats], {}) == [0, 1, 2]
127

```

20. assistant (2026-06-18T07:55:09.226Z)

Motion segmenter has NO direct test. Let me confirm that and check the `\_\_init\_\_.py` and confirm there's no indirect coverage of motion's process_video. Let me also check whether gemini has helper functions that could be shared, and examine the `\_\_init\_\_.py`.

21. user (2026-06-18T07:55:11.290Z)

```
tests\test_async_jobs.py:215:    mp setattr(m.segmenter_registry, "list_available", lambda:
{"motion": "..."})
tests\test_calibrate_local.py:27:    gated = pf((0.02, 2.0, 1.0, 3))          # monotonic
motion crosses net <3x -> gated out
tests\test_config.py:235:        json.dump({"indexing": {"default_segmenter": "motion"}},
fh)
tests\test_config.py:237:        assert cfg.indexing.default_segmenter == "motion"
tests\test_config.py:250:        "indexing": {"default_segmenter": "motion"},
tests\test_config.py:255:        assert cfg.indexing.default_segmenter == "motion"
tests\test_database.py:138:    assert db.query_candidate_segments("vid1", detector="motion") ==
[]
tests\test_database.py:305:        "fusion_config", "classifier_json", "promotion_evidence",
tests\test_database.py:317:        promotion_evidence={"promote": True, "n": 6},
tests\test_database.py:330:    assert m["promotion_evidence"]["promote"] is True
tests\test_database.py:400:    assert m["promotion_evidence"] is None
tests\test_eval_batch.py:25: def test_auto_stage_final_only_for_motion():
tests\test_eval_batch.py:26:     assert batch.default_stages_for("motion", "auto") == ["final"]
tests\test_eval_batch.py:35:     assert batch.default_stages_for("motion", "both") ==
["candidates", "final"]
tests\test_fusion_features.py:70: #
----- mean_player_motion
tests\test_fusion_features.py:72: def test_mean_player_motion():
tests\test_fusion_features.py:75:     assert ff.mean_player_motion(per_frame, 100, 100) ==
pytest.approx(0.10)
tests\test_fusion_features.py:77:     assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}},
100, 100) == 0.0
tests\test_fusion_features.py:78:     assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1:
[(2, (10, 0, 20, 10))]}}, 100, 100) == 0.0
tests\test_fusion_features.py:81: #
----- two_sided_motion
tests\test_fusion_features.py:83: def test_two_sided_motion():
tests\test_fusion_features.py:88:     assert ff.two_sided_motion(both, 100, 100, net, None) ==
pytest.approx(0.5)
tests\test_fusion_features.py:90:     assert ff.two_sided_motion(one_sided, 100, 100, net, None)
== 0.0
tests\test_fusion_features.py:91:     assert ff.two_sided_motion(both, 100, 100, None, None) ==
0.0 # no net -> 0
tests\test_fusion_features.py:94: def test_two_sided_motion_respects_court_polygon():
tests\test_fusion_features.py:99:     assert ff.two_sided_motion(per_frame, 100, 100, net, None)
== 1.0 # both sides present
tests\test_fusion_features.py:101:     assert ff.two_sided_motion(per_frame, 100, 100, net,
LEFT_HALF) == 0.0
tests\test_fusion_features.py:140:     assert feats["two_sided_motion"] == pytest.approx(1.0) #
both sides occupied every frame
tests\test_fusion_hybrid.py:74:     # The 4 base motion keys are still there.
tests\test_fusion_hybrid.py:110:     # glue (in-court counts -> players_in_court; track
displacement -> mean_player_motion),
tests\test_fusion_hybrid.py:126:     assert stats[0]["mean_player_motion"] > 0.0 # both
tracks moved -> non-zero motion
tests\test_personalization_serving.py:72:     """A model fit below the promotion floor
(n_videos_at_fit < min_n) is not trusted to override."""
tests\test_person_detector.py:5: the motion-window extraction that feeds the candidate `stats` +
the learned fusion features.
tests\test_promotion.py:1: """Tests for backend.eval.promotion ? the eval?serve promotion guard.
tests\test_promotion.py:10: from backend.eval.promotion import (
tests\test_promotion.py:39:     cannot carry a promotion.
tests\test_promotion.py:70:     """vetoed_by_served distinguishes 'guard overruled a would-be
promotion' from 'never qualified'.
tests\test_per_user_gate.py:1: """Tests for backend.eval.per_user_gate ? the per-user
personalization promotion gate.
tests\test_per_user_gate.py:13: from backend.eval.per_user_gate import PromotionDecision,
should_promote_for_user
tests\test_per_user_gate.py:21:     assert isinstance(d, PromotionDecision)
```

```

tests\test_rally_seg_eval.py:82:# --- R1: compare() surfaces the net over-seg promotion guard
----- #
tests\test_reporting_harvest.py:134:def
test_motion_segmenter_flags_synthetic_metadata(monkeypatch):
tests\test_reporting_harvest.py:136:     env = _record(segmenter_requested="motion",
segmenter_actual="motion",
tests\test_reporting_harvest.py:137:         config={"indexing": {"default_segmenter":
"motion"}})
tests\test_reporting_harvest.py:139:     assert "synthetic_motion_metadata" in codes
tests\test_reporting_harvest.py:140:     # motion is not AI-based -> the provider-noop rule must
NOT fire even with no key
tests\test_reporting_harvest.py:227:         segmenter_actual="motion", segmentation_ran=True,
tests\test_results.py:43:     ``run_phase3_eval`` defaults to the ``motion`` segmenter (a real
default-path crash);
tests\test_results.py:44:     ``run_process_and_stitch`` hits the same path whenever it's
configured to use ``motion``.
tests\test_serve_contrast.py:5: - run_contrast feeds those into the served-guarded promotion
gate;
tests\test_tolerance_metrics.py:120:# R1 ? net over-segmentation promotion guard
tests\test_trajectory_hybrid_equivalence.py:106:def
test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
tests\test_trajectory_hybrid_equivalence.py:117:     """Directly pin the two fusion seams on the
base: stats = the 4 motion keys verbatim,
tests\test_tracknet_runner.py:101:     # Two bursts of motion separated by a long invisible gap
=> two windows.
tests\test_tracknet_runner.py:207:     # one window of fast motion then slow drift (both visible
& > velocity_thresh ? one group).
tests\test_windowing.py:24:     # 0-8: short fast burst (~0.8s of motion) ? survives proposal,
dropped by served (min 2s).
tests\test_windowing.py:32:     # 30-80: long fast rally (~5s of motion) ? survives BOTH presets.
tests\test_yolo_hybrid.py:165:     """The person cue's window extraction should return dicts with
motion stats."""

```

22. user (2026-06-18T07:55:11.884Z)

```

1     """TrackNet-based hybrid play segmenter.
2
3     Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4     in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5     tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6     purpose-built trajectory model localises it far more precisely, giving cleaner
7     candidate rallies than person-velocity heuristics.
8
9     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10    lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11    TrackNet runner and identity.
12    """
13    from typing import Any, Dict, Tuple
14
15    from backend.pipeline.segmenters.base import segmenter_registry
16    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
17    from backend.pipeline.detectors.base import DetectorRunner
18    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
19
20
21    class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
22        family = "tracknet"
23        display_label = "TrackNet"
24
25        @property
26        def name(self) -> str:
27            return "tracknet_hybrid"
28
29        @property
30        def description(self) -> str:

```



```

31         return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
32             "candidate rallies + AI provider for high-precision semantic
boundaries.")
33
34     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
35         cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
36         return TrackNetRunner(cfg), {
37             "weights_path": cfg.weights_path,
38             "queue_length": cfg.queue_length,
39         }
40
41
42     segmenter_registry.register(TrackNetHybridSegmenter)
43

```

23. assistant (2026-06-18T07:55:15.120Z)

Now let me check the `test\_reporting\_harvest.py` motion test to see if it actually exercises `process\_video`, and confirm motion's process_video has no direct test. Let me also check the fusion_hybrid to understand the inheritance picture for the cluster.

24. user (2026-06-18T07:55:17.132Z)

```

125         was_reingest=False, segmentation_ran=True,
126     )
127     assert paths is not None
128     with open(paths["json"], encoding="utf-8") as f:
129         assert "oversize_clip_skipped" in f.read()
130     # Consumed: a second pop yields nothing (no cross-run leakage).
131     assert run_signals.pop("md5sig") == {}
132
133
134     def test_motion_segmenter_flags_synthetic_metadata(monkeypatch):
135         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
136         env = _record(segmenter_requested="motion", segmenter_actual="motion",
137             config={"indexing": {"default_segmenter": "motion"}})
138         codes = _codes(env)
139         assert "synthetic_motion_metadata" in codes
140         # motion is not AI-based -> the provider-noop rule must NOT fire even with no key
141         assert "silent_provider_noop" not in codes
142
143
144     def test_segmenter_divergence_fires_only_without_explicit_override(monkeypatch):
145         monkeypatch.setenv("GEMINI_API_KEY", "x")
146         cfg = {"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}}
147         # No explicit request, but a different segmenter ran -> a fallback diverged: flag
148         it.
149         env = _record(segmenter_requested=None, segmenter_actual="gemini", config=cfg)
150         assert "segmenter_divergence" in _codes(env)

```

25. user (2026-06-18T07:55:17.697Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9       GPU-free, computed from the trajectory we already have); and, only when the person cue
10       is explicitly enabled (`indexing.fusion.cue_flags.person`), the person-cue features

```

```

11     (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13     torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14     model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15     via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16     - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
experiment
20     harness can re-score any variant.
21
22     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54         @property
55         def name(self) -> str:
56             return "fusion_hybrid"
57
58         @property
59         def description(self) -> str:
60             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "

```

```

61         "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62         "AI provider sets semantic boundaries.")
63
64     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66         if substrate == "tracknet":
67             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68             return TrackNetRunner(cfg), {
69                 "weights_path": cfg.weights_path,
70                 "queue_length": cfg.queue_length,
71                 "fusion_substrate": "tracknet",
72             }
73         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                frame_width: float, video_path: str,
78                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83         gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
84                                              min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92     # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93     # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94     # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95     # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96     # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97     # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98     # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100    def _axis_estimate(self, points: List[Any]) -> tuple:
101        """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102        computed once per video, not once per candidate window (it depends only on
points)."""
103        key = id(points)
104        if self._axis_cache_key != key or self._axis_cache is None:
105            self._axis_cache_key = key
106            self._axis_cache = rally_gate.estimate_play_axis(points)
107        est = self._axis_cache
108        assert est is not None

```

```

109         return est
110
111     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                          frame_width: float, video_path: str,
113                          fcfg: Dict[str, Any]) -> Dict[str, float]:
114         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115         frame range and compute the scale/translation-invariant person features. Lazily
builds
116         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118         from backend.pipeline.detectors.person_detector import PersonDetector
119         if self._person is None:
120             self._person = PersonDetector(self.config)
121
122         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127         fw = det["frame_width"] or frame_width
128         fh = det["frame_height"]
129         # If we can't trust the frame dims, every normalised feature would silently
collapse
130         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131         if fw <= 0 or fh <= 0:
132             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134         poly = fcfg.get("court_polygon")
135         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                         court_polygon=polygon, net=net)
140
141     def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                            window_stats: List[Dict[str, Any]],
143                            idx_cfg: Dict[str, Any]) -> List[int]:
144         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145         (0) keep every window ? identical to the substrate. Persisted candidates are
146         untouched (full superset).
147
148         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
149         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
150         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
151         set rather than emit a video with zero rallies. This makes Gate A/B strictly
152         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
153         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
154         below it always wins on an otherwise-empty video. Observed on real footage
155         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157         fcfg = idx_cfg.get("fusion", {})
158         min_crossings = int(fcfg.get("min_crossings", 0))
159         min_players = int(fcfg.get("min_players", 0))

```

```

160         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162         keep: List[int] = []
163         for i, st in enumerate(window_stats):
164             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                 continue
166             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                 continue
168             keep.append(i)
169
170         if not keep and window_stats:
171             # Gating emptied a non-empty video ? fall back to the ungated set.
172             return list(range(len(window_stats)))
173         return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177

```

26. assistant (2026-06-18T07:55:22.771Z)

The ``test_motion_segmenter_flags_synthetic_metadata`` test exercises reporting, NOT motion's ``process_video`` ? it doesn't call ``process_video`` at all. So motion's ``process_video`` (204 LOC) has zero direct test coverage. Combined with ``random.sample``/``random.randint`` non-determinism inside, it's fundamentally hard to characterize.

Now let me verify the test suite runs for the segmenters I'm considering, and check the duplicated provider-wiring block across `gemini/yolo_hybrid` to assess a shared-helper extraction.

27. user (2026-06-18T07:55:32.997Z)

```

..... [100%]
30 passed in 6.62s

```

28. assistant (2026-06-18T07:55:39.741Z)

All 30 tests green. Now let me verify the exact duplication of the provider-wiring preamble between `gemini` and `yolo_hybrid` to assess whether a shared helper is a clean, behavior-preserving dedup. The key question is whether the blocks are truly identical (same log strings, same return semantics).

The `gemini` block (lines 41-68) and `yolo_hybrid` block (lines 60-88) both do: sport lookup ? provider/model name ? API key env ? provider get. But there's a subtle difference: `gemini` does sport lookup **first** then API key; `yolo_hybrid` does sport lookup first too. However, ``trajectory_hybrid`` already inlines its own variant with ``skip_ai``. Let me check the exact return-on-failure semantics and whether extraction would change behavior (e.g. error log ordering).

29. user (2026-06-18T07:55:41.290Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Phase 2 uses WASB (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4     WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5     badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7     For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8     candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9     min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.

```

```

14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40
41     segmenter_registry.register(WasbHybridSegmenter)
42

```

30. user (2026-06-18T07:55:42.918Z)

```

192:         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
201:         if isinstance(result, Failure):
213:             segments = result.value if hasattr(result, "value") else result
347: def process_video(req: ProcessRequest):
835:         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
838:         if isinstance(result, Failure):
843:             segments = result.value if hasattr(result, "value") else result

```

31. assistant (2026-06-18T07:55:49.096Z)

This confirms the critical contract: `main.py` handles BOTH a bare list AND a `Result` (`isinstance(result, Failure)` then `result.value if hasattr(result, "value") else result`). So gemini/yolo_hybrid/trajectory_hybrid return bare lists; motion returns `Success`/`Failure`. Any extraction must preserve the exact return type per segmenter.

I now have enough to assess each proposal. Let me look at the `\_select\_for\_handoff`/`handoff` inner-function structure once more and check the gemini chunked summary block ? that summary-logging is a clean pure extraction target. Let me also verify there's no hidden shared base helper opportunity by checking what's already in base.py (already read ? it's minimal ABC).

Let me check the `.coveragerc` floor and whether these files are in coverage scope.

32. user (2026-06-18T07:55:49.905Z)

```

1     [run]
2     # Excluded from coverage: the legacy motion segmenter (demo baseline with
3     # fabricated metadata) and the concrete validators (thin OpenCV/ffprobe

```

```

4      # wrappers only exercisable against real media files). validators/base.py
5      # stays counted ? it has a dedicated unit suite.
6      omit =
7          backend/pipeline/segmenters/motion.py
8          backend/pipeline/validators/format.py
9          backend/pipeline/validators/resolution_fps.py
10         backend/pipeline/validators/pts_continuity.py
11         backend/pipeline/validators/scene_cut.py
12         backend/pipeline/validators/blur.py
13         backend/pipeline/validators/relevance.py
14

```

33. assistant (2026-06-18T07:56:01.718Z)

This confirms motion.py is explicitly OMITTED from coverage and labeled a "legacy demo baseline with fabricated metadata." That's a strong signal: motion's `process\_video` should NOT be refactored ? needs characterization tests first, and the project itself treats it as throwaway.

I now have a complete picture. Let me synthesize my findings.

****Summary of what I found:****

1. ****gemini.py `process\_video` (434L)**** ? well-covered by `tests/test\_gemini.py` (8 tests: single-file, chunked, re-encode, scale-width, oversize loud-skip both paths). Two clean extraction targets: the inner `parse\_time` helper (already a nested fn, pure) and the two summary-log blocks. Returns bare list.
2. ****yolo_hybrid.py `process\_video` (309L)**** ? well-covered by `tests/test\_yolo\_hybrid.py` (7 tests incl. output equivalence, candidate persist/link, store-disabled, bad-timestamp drop). Stage-1 already extracted to `PersonDetector`. Returns bare list.
3. ****trajectory_hybrid.py (231L)**** ? ALREADY refactored into a base class with hooks; `wasb\_hybrid`/`tracknet\_hybrid` are thin subclasses; strong equivalence test suite. No further refactor warranted.
4. ****motion.py `process\_video` (204L)**** ? NO direct test, explicitly OMITTED from coverage, labeled "legacy demo baseline with fabricated metadata," and riddled with `random.\*` non-determinism. Must NOT be refactored without characterization tests.
5. ****Shared structure to base.py****: the provider-wiring preamble (sport lookup ? provider/model name ? API-key env ? `provider\_registry.get`) is near-duplicated across gemini, yolo_hybrid, and trajectory_hybrid. A shared helper is *possible* but risky ? the three variants differ subtly (trajectory has `skip\_ai`; error-return types differ) and the dedup spans 3 files, making it a larger/riskier PR. I'll propose it at lower value/higher risk.

Let me record the chapter and return the structured output.

34. user (2026-06-18T07:56:02.558Z)

Chapter marked: "Segmenter refactor scoping". Continue your current work.

35. user (2026-06-18T07:56:54.852Z)

Structured output provided successfully

36. assistant (2026-06-18T07:57:02.189Z)

Scoping complete. I read the four target segmenters and their tests, verified a green 30/30 baseline, and returned four proposals via StructuredOutput.

Key findings:

- ****gemini.py**** and ****yolo_hybrid.py**** ? safe high-value/low-risk intra-file extract-method decompositions; both have strong mocked orchestration tests (the yolo one includes an explicit output-equivalence regression guard).

- `**trajectory_hybrid.py**` ? already the refactored hook-based base class (wasb/tracknet are thin subclasses with an equivalence suite); no further work warranted.
- `**motion.py**` ? flagged DO-NOT-REFACTOR: zero direct tests, explicitly omitted in `coveragerc` as a legacy demo, and random-saturated; needs characterization tests first.
- The shared provider/API-key preamble dedup is possible but medium-risk (cross-file, touches `base.py`, call sites differ on `skip_ai`).
- Load-bearing contract noted: `backend/main.py` consumes both bare lists and `Result` objects, so every extraction must preserve each segmenter's existing return type (motion returns `Success`/`Failure`; the other three return bare lists).